作业 6：加速结构实验报告

姓名：刘博文
学号：09023225

1 实验背景与目的

在前序作业中，我们已经完成了基于 Whitted 风格的基础光线追踪器，实现了光线生成、光线与球体及三角形的相交检测以及简单的光照模型。然而，该实现采用的是最直接的求交方式：对每一条光线遍历场景中的所有物体并逐一测试是否发生相交。当场景规模较小时，该方法尚能接受，但随着物体数量和三角形数量的增加，求交开销将呈线性增长，严重制约渲染效率。

为解决上述问题，本实验引入加速结构 Bounding Volume Hierarchy（BVH，包围盒层次结构），通过在空间层次上对物体进行组织，从而显著减少光线与物体求交时的无效测试次数。实验的主要目标包括：实现光线与轴对齐包围盒（AABB）的相交检测算法，以及在此基础上完成 BVH 的递归遍历求交过程，并验证加速结构在复杂场景中的正确性和有效性。

2 实验原理

BVH 是一种层次化的空间加速结构。其基本思想是：使用简单几何体（通常为轴对齐包围盒）对场景中复杂物体或物体集合进行包裹，并递归地将物体集合划分为左右子集，构成一棵二叉树结构。BVH 的每个节点包含一个包围盒，用于近似表示该节点下所有物体的空间范围。

在光线追踪过程中，当一条光线与某一 BVH 节点的包围盒不相交时，该节点下的所有物体均可被整体剔除；只有当光线与包围盒发生相交时，才继续向子节点递归，从而大幅减少不必要的光线-物体相交测试。

2.1 光线与包围盒相交（Ray–AABB Intersection）

实验中采用的是基于 slab 方法的光线与轴对齐包围盒求交算法。对于每一个坐标轴，分别计算光线进入和离开该轴对应平面的参数区间，并取三个轴方向区间的交集。当最终区间非空且与光线的有效传播区间重叠时，即判定光线与包围盒发生相交。

为提高计算效率，预先计算光线方向的倒数向量 `invDir`，并通过 `dirIsNeg` 数组记录光线在各轴方向上的符号，从而避免在求交过程中频繁进行除法和条件判断。

2.2 BVH 遍历与求交

BVH 遍历过程采用递归方式实现。从根节点开始，首先判断光线是否与当前节点的包围盒相交；若不相交，则直接返回空交点结果；若相交，则分情况处理：当节点为叶节点时，直接对其所包含的物体执行精确求交；当节点为内部节点时，递归访问左右子节点，并返回距离光线起点最近的有效交点。

该过程确保了在保证求交正确性的前提下，最大限度地减少光线与无关物体的相交测试次数。

3 实验实现

3.1 光线与包围盒相交函数的实现

在 `Bounds3::IntersectP` 函数中，根据光线方向和包围盒的最小、最大顶点，分别计算光线在三个坐标轴方向上的进入参数 $t_{\min}$ 与离开参数 $t_{\max}$。随后对三个方向的区间进行合并，并与光线本身的有效区间 $[t_{\min}^{ray}, t_{\max}^{ray}]$ 进行比较。当两个区间存在重叠时，返回相交结果。

```
inline bool Bounds3::IntersectP(const Ray& ray, const Vector3f& invDir,
                                const std::array<int, 3>& dirIsNeg) const
{
    // invDir: ray direction(x,y,z), invDir=(1.0/x,1.0/y,1.0/z), use this because Multiply is faster that Division
    // dirIsNeg: ray direction(x,y,z), dirIsNeg=[int(x>0),int(y>0),int(z>0)], use this to simplify your logic
    // TODO test if ray bound intersects
    // pMin / pMax 是包围盒的两个角点
    const Vector3f& boundsMin = pMin;
    const Vector3f& boundsMax = pMax;
    // x 方向
    float tMin = ( (dirIsNeg[0] ? boundsMax.x : boundsMin.x) - ray.origin.x ) * invDir.x;
    float tMax = ( (dirIsNeg[0] ? boundsMin.x : boundsMax.x) - ray.origin.x ) * invDir.x;
    // y 方向
    float tyMin = ( (dirIsNeg[1] ? boundsMax.y : boundsMin.y) - ray.origin.y ) * invDir.y;
    float tyMax = ( (dirIsNeg[1] ? boundsMin.y : boundsMax.y) - ray.origin.y ) * invDir.y;
    // 若在 y 方向没有重叠，直接返回 false
    if (tMin > tyMax || tyMin > tMax)
        return false;
    // 更新 tMin, tMax 为当前区间交集
    if (tyMin > tMin) tMin = tyMin;
    if (tyMax < tMax) tMax = tyMax;
    // z 方向
    float tzMin = ( (dirIsNeg[2] ? boundsMax.z : boundsMin.z) - ray.origin.z ) * invDir.z;
    float tzMax = ( (dirIsNeg[2] ? boundsMin.z : boundsMax.z) - ray.origin.z ) * invDir.z;

    if (tMin > tzMax || tzMin > tMax)
        return false;

    if (tzMin > tMin) tMin = tzMin;
    if (tzMax < tMax) tMax = tzMax;
    // 同时满足 ray 的有效范围
    if (tMax < ray.t_min || tMin > ray.t_max)
        return false;
    return true;
}
```

图 1: 具体的代码实现

该实现避免了分支复杂度过高的问题，并在数值稳定性方面满足实验要求。

3.2 BVH 加速结构求交函数的实现

在 `BVHAccel::getIntersection` 函数中，首先判断光线是否与当前节点的包围盒相交；若不相交，直接返回空的交点结果。若相交且当前节点为叶节点，则遍历该节点中存储的物体列表，返回最近的交点信息。若当前节点为内部节点，则递归访问左右子节点，并比较左右子树返回的交点距离，最终返回最近的有效交点。

```cpp
Intersection BVHAccel::getIntersection(BVHBuildNode* node, const Ray& ray) const
{
    // TODO Traverse the BVH to find intersection
    Intersection inter;

    if (!node)
        return inter;

    // 预计算 invDir 与 dirIsNeg（Bounds3::IntersectP 需要）
    Vector3f invDir(1.0f / ray.direction.x,
                    1.0f / ray.direction.y,
                    1.0f / ray.direction.z);

    std::array<int, 3> dirIsNeg = {
        invDir.x < 0,
        invDir.y < 0,
        invDir.z < 0
    };

    // 若光线与当前节点包围盒不相交，直接返回
    if (!node->bounds.IntersectP(ray, invDir, dirIsNeg))
        return inter;

    // 叶子节点：与物体求交
    if (node->left == nullptr && node->right == nullptr)
    {
        return node->object->getIntersection(ray);
    }

    // 非叶子节点：递归左右子树
    Intersection hitLeft  = getIntersection(node->left,  ray);
    Intersection hitRight = getIntersection(node->right, ray);

    // 返回更近的交点
    if (hitLeft.happened && hitRight.happened)
        return (hitLeft.distance < hitRight.distance) ? hitLeft : hitRight;
    else if (hitLeft.happened)
        return hitLeft;
    else
        return hitRight;
}
```

图 2: 具体的代码实现

该实现严格遵循 BVH 的递归遍历逻辑，保证了算法的正确性和一致性。

4 实验结果与分析

在完成包围盒求交与 BVH 遍历逻辑后，程序能够在包含大量三角形的场景中正确渲染出几何形体，并且相较于不使用加速结构的实现，渲染效率有明显提升。

```
cs18@games101vm:~/Desktop/Assignment6/build$ ./RayTracing
BVH Generation complete:
Time Taken: 0 hrs, 0 mins, 0 secs

 - Generating BVH...

BVH Generation complete:
Time Taken: 0 hrs, 0 mins, 0 secs

Render complete: ==================================================] 100 %
Time taken: 0 hours
          : 0 minutes
          : 1 seconds
```

图 3: 渲染花费的时间

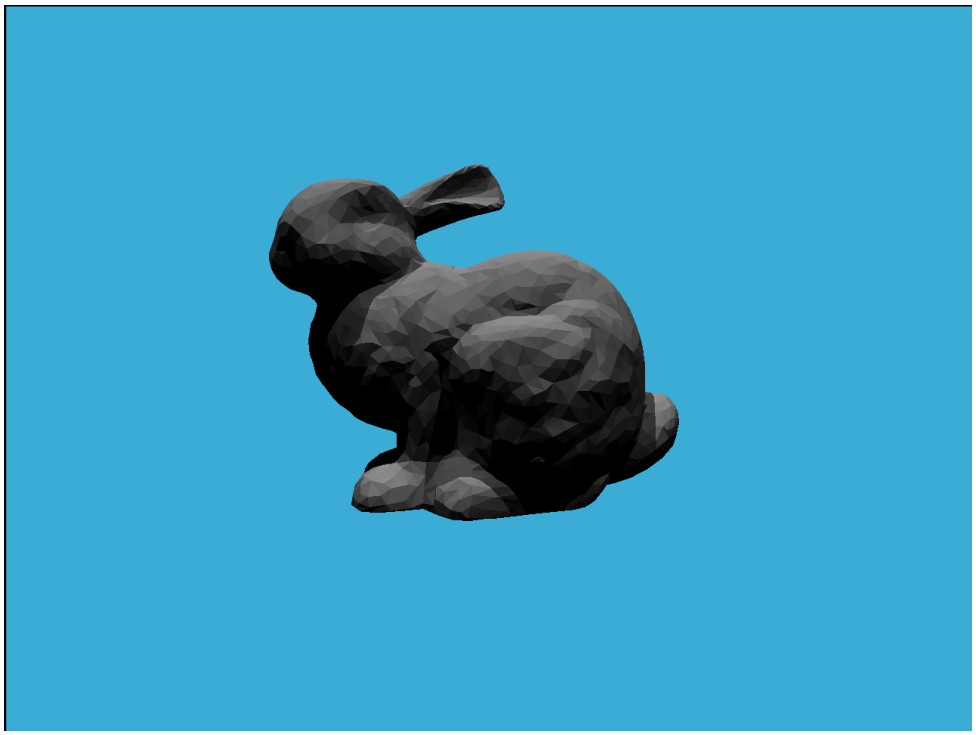

图 4: 使用 BVH 加速结构渲染得到的实验结果图像

从渲染结果可以观察到，场景中的物体边界清晰，遮挡关系正确，说明 BVH 求交过程在数值和逻辑上均未引入错误。同时，在调试输出和运行时间对比中可以发现，当场景复杂度提高时，BVH 能够有效减少光线与物体的求交次数，从而加速整体渲染过程。

5 实验总结

通过本次实验，我系统地理解并实现了 Bounding Volume Hierarchy 加速结构的核心原理与关键算法，包括光线与轴对齐包围盒的求交方法以及 BVH 的递归遍历求交过程。实验结果表明，引入 BVH 后可以显著提升光线追踪在复杂场景下的运行效率。

本实验不仅加深了我对光线追踪整体流程的理解，也使我对空间数据结构在图形学中的重要作用有了更加直观的认识，为后续进一步学习更高级的加速结构与渲染算法奠定了基础。